

Wireshark MCP

Packet Analysis and Network Forensics

Technical Documentation

Ritvik Indupuri

1 Project overview

Wireshark MCP makes offline packet-capture analysis available through the Model Context Protocol. An analyst can use an MCP-compatible assistant to inspect captures, identify communication patterns, examine DNS and TLS metadata, search for exposed credentials, and retrieve conversational payloads. The server returns structured evidence for the client to interpret.

The application is a Python package named `wireshark_mcp`, version 0.1.0. FastMCP exposes ten tools over standard input and output. Scapy provides capture parsing and the forensic analyzers; an optional TShark executable adds native display filtering and protocol hierarchy statistics.

Component	Responsibility
MCP client	Select tools and interpret results. Model selection and natural-language reasoning belong to the client.
<code>server.py</code>	Register tools, check file existence, process uploads and compose automated triage.
ScapyEngine	Read capture metadata and aggregate bidirectional conversations.
TsharkEngine	Run Wireshark command-line analysis with subprocess timeouts.
Forensic analyzers	Compute DNS statistics, TLS fingerprints, credential/scan indicators and stream transcripts.
Local storage	Retain uploaded capture files for subsequent tool calls.

Execution boundary

The server does not contain an LLM SDK, model API key, autonomous agent loop or model-specific prompt chain. Its tools are deterministic Python functions. The agent icon in the diagrams identifies the external AI client only. Server instructions supply usage examples; those examples do not add tool implementations.

Analysis operates on existing files or decoded uploads. The registered tools do not perform live packet capture, download sample traffic or execute remediation. Capture acquisition can occur separately before the file is submitted.

Reading guide

Sections 2-4 describe the architecture, tool interface and automated upload flow. Sections 5-8 explain capture statistics, DNS analysis, threat heuristics, TLS fields and streams. Sections 9-10 cover setup, operating behavior, source references and next steps.

2 System architecture

The client invokes an MCP tool with a capture path or encoded upload. The server selects the required local analysis function and returns its evidence through the same stdio session.

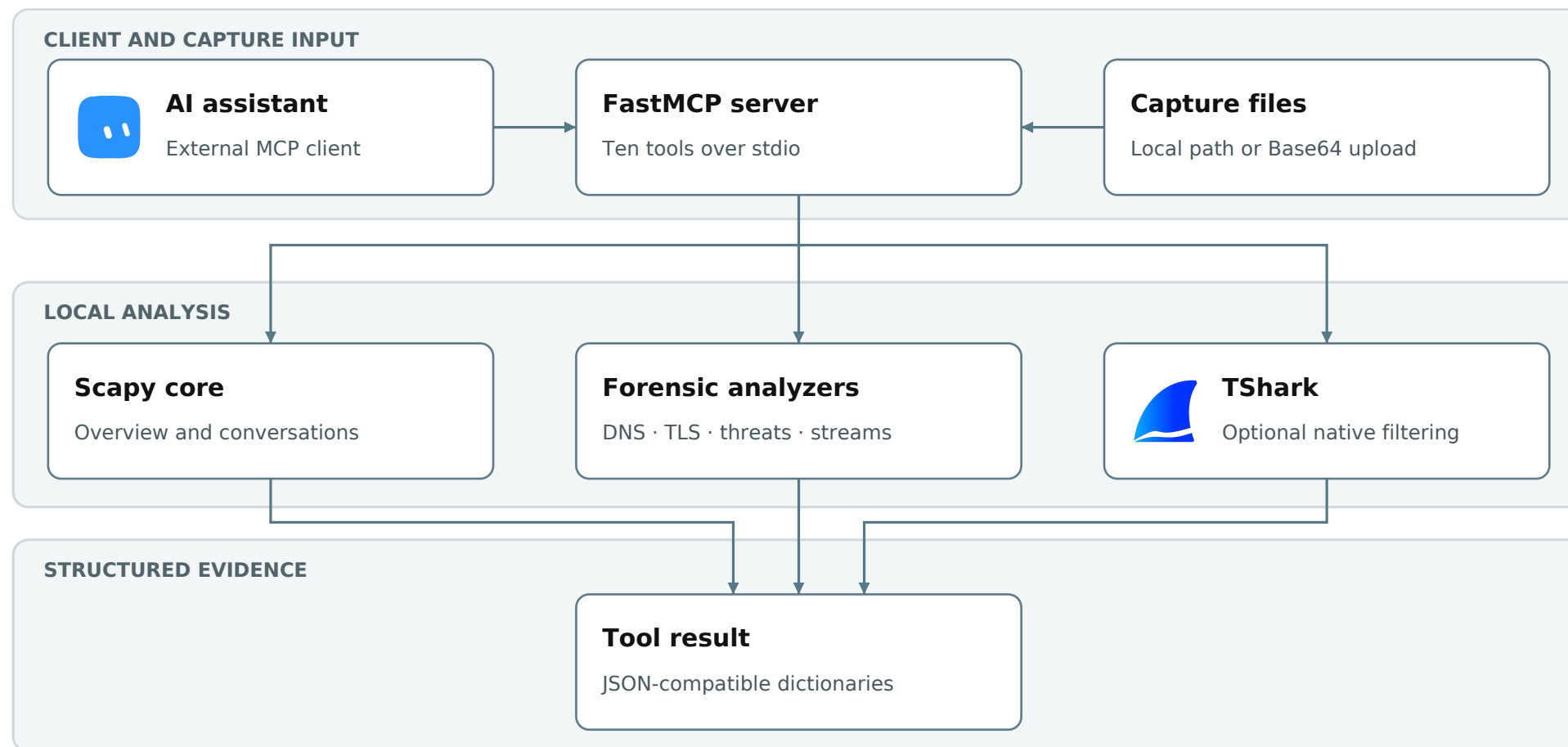


Figure 1: MCP client, capture inputs and local packet-analysis components.

The branches show component routing, not parallel execution. Forensic analyzers also read packets through Scapy. TShark is optional and is invoked for display filters and supplementary overview statistics. Results return to the calling client; the server does not send them directly to a model provider.

3 MCP tool interface

All file-based tools accept `file_path` and check that it identifies an existing file. Paths are resolved on the server host. Tool annotations describe argument types; no custom Pydantic request models or numeric range constraints are declared in this package.

Tool	Arguments and result
<code>get_engine_status</code>	No arguments. Returns discovery status, TShark path, capability descriptions and example queries.
<code>pcap_overview</code>	<code>file_path</code> . Returns file size, packet count, duration, timestamps and protocol counts; optionally native hierarchy text.
<code>list_conversations</code>	<code>file_path</code> ; <code>protocol="ip"</code> . Supports ip, tcp or udp grouping. Returns total count and the top 50 pairs by bytes.
<code>apply_display_filter</code>	<code>file_path</code> , <code>display_filter</code> ; <code>limit=50</code> , <code>offset=0</code> . Requires TShark. Returns paginated packet fields and match counts.
<code>extract_dns_forensics</code>	<code>file_path</code> ; <code>entropy_threshold=3.6</code> , <code>subdomain_len_threshold=25</code> . Returns DNS counts and flagged names.
<code>extract_tls_fingerprints</code>	<code>file_path</code> . Returns ClientHello observations, SNI counts and implementation-specific fingerprint hashes.
<code>scan_suspicious_indicators</code>	<code>file_path</code> . Returns credential and SYN-scan findings, severity counts and a separate beaconing list.
<code>follow_stream</code>	<code>file_path</code> ; <code>stream_index=0</code> , <code>protocol="tcp"</code> , <code>max_payload_bytes=65536</code> . Returns an ordered payload transcript.
<code>upload_pcap</code>	<code>base64_data</code> ; optional filename. Stores capture and returns path, filename, success message and quick overview.
<code>analyze_uploaded_pcap</code>	<code>base64_data</code> ; optional filename. Uploads and combines overview, conversations, threats, DNS and TLS results.

Response conventions

Tools return dictionaries that FastMCP exposes to the client. Some expected conditions return an error field, including missing TShark, rejected uploads and an out-of-range stream index. Missing files raise `FileNotFoundError`. Parsing and subprocess failures can propagate through the tool call.

The automated triage result contains `file_path`, `filename`, `overview`, `conversations`, `threat_findings`, `dns_forensics` and `tls_fingerprints`. Each nested analysis retains its own counters and output limits.

4 Upload and analysis lifecycle

`analyze_uploaded_pcap` composes existing tools in a fixed sequence. The external assistant chooses when to call it; the server performs the following steps without a model call.

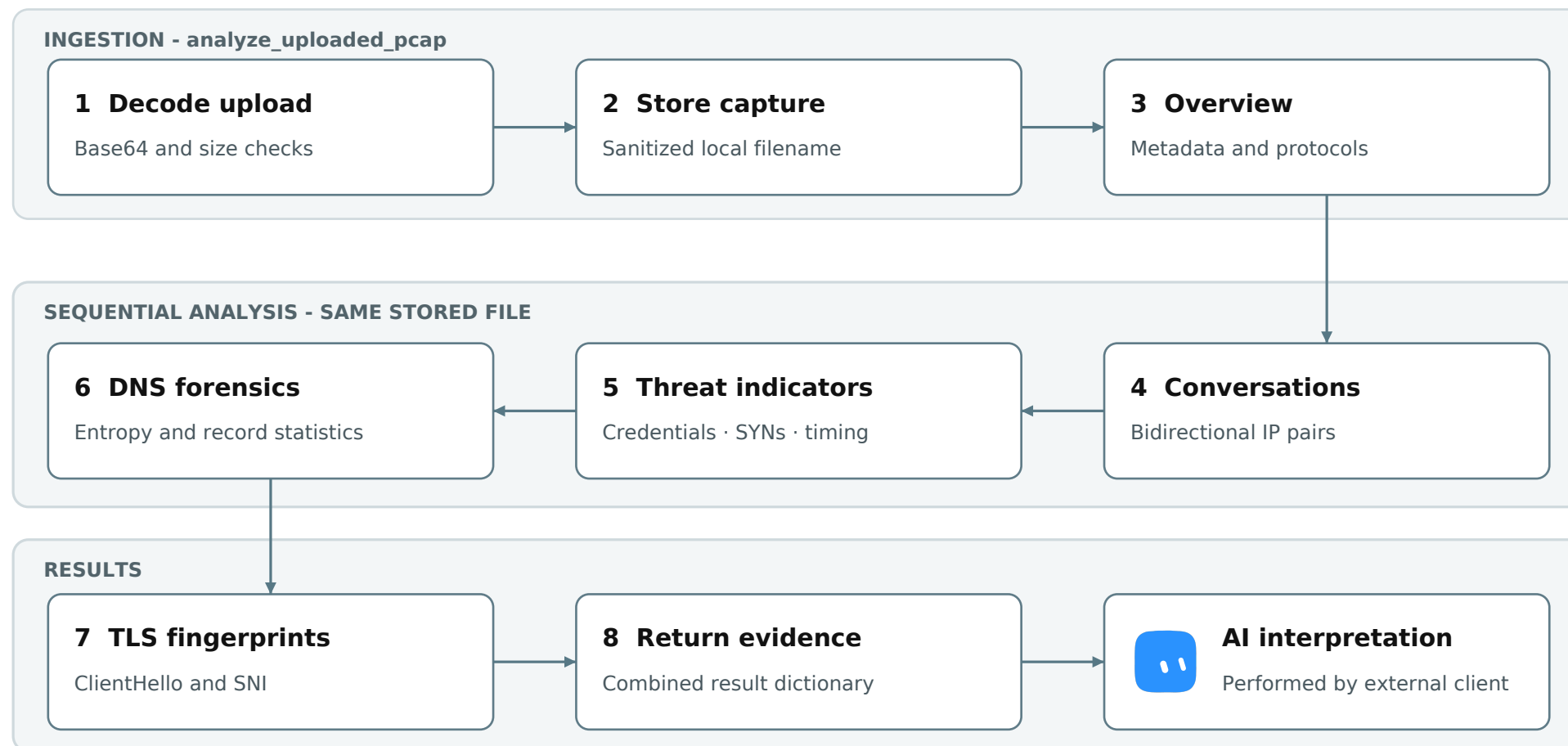


Figure 2: Stored upload, sequential forensic analysis and client-side interpretation.

`upload_pcap` performs the first three steps and supplies the overview to the combined report. The remaining calls reread the same file in sequence. Filtering and stream following are separate follow-up tools. A later analysis error interrupts the combined call; there is no per-stage recovery wrapper.

5 Capture ingestion and packet statistics

Upload processing

The upload directory is `wireshark_mcp_uploads` beneath the `TEMP` environment variable, or beneath the working directory when `TEMP` is absent. It is created when `server.py` is imported. The encoded input is checked against 1.4 times a 50 MiB limit; decoded data is then limited to $50 \times 1024 \times 1024$ bytes.

The filename passes through `basename`, removal of dot-dot and slash characters, and an allowlist of alphanumeric characters plus period, underscore and hyphen. Missing or empty names receive a random `upload_` prefix. A `.pcap` suffix is appended unless the sanitized name ends in `.pcap`, `.pcapng` or `.cap`. The file is written before its quick overview is parsed.

Uploads with the same sanitized name overwrite the existing file. Storage is shared by the server process, with no per-user directory or automatic deletion. The decoded bytes are not validated as a capture before writing; Scapy performs parsing during the overview.

Overview calculation

Scapy reads the capture with `rdpcap`. The overview reports disk size, packet count, first and last packet timestamps, and their nonnegative duration difference. Timestamps are formatted in the host local timezone. Empty captures return zero packets and duration, null timestamps and an empty protocols dictionary; nonempty captures use `protocol_breakdown`.

Each packet receives one primary classification: TCP, UDP, ICMP, ARP, IPv6, other IPv4 or Other. DNS is counted additionally when decoded; HTTP and TLS are inferred from raw payload prefixes. Counts are therefore overlapping observations, not percentages that necessarily sum to the packet count.

Conversation aggregation

IP grouping combines IPv4 or IPv6 endpoint pairs. TCP and UDP grouping add ports. Sorting endpoint strings creates a canonical bidirectional pair; packet lengths and counts are accumulated in capture order, while the earliest and latest timestamps define duration. Results are sorted by `total_bytes` in descending order and the server returns at most 50 rows.

Conversation rows contain `endpoint_a`, `endpoint_b`, `total_packets`, `total_bytes` and `duration_seconds`. The `total_conversations` field counts all grouped pairs, including those beyond the returned slice. These are endpoint-pair aggregates rather than connection-state records.

6 DNS forensics

The DNS analyzer reads decoded DNS layers, counts question names and record types, records response codes, and examines names for high entropy or excessive subdomain length. The output supports analyst triage of possible tunneling or algorithmically generated names.

Entropy and name selection

For a lowercased string, Shannon entropy is $H = -\sum(p \times \log_2(p))$, where p is each character's frequency divided by string length. Empty strings have entropy zero. The analyzer calculates entropy for the complete name and for the selected subdomain, rounding both to three decimals.

For names with more than two labels, the subdomain is everything before the last two labels. Otherwise the first label is used. This is a simple label split, not a public-suffix calculation. Each distinct name is evaluated once for suspiciousness.

Condition	Implemented rule
Entropy flag	Rounded subdomain entropy $\geq$ entropy_threshold (default 3.6), with at least 12 characters.
Length flag	Subdomain length $\geq$ subdomain_len_threshold (default 25).
Response statistics	Counts NOERROR, FORMERR, SERVFAIL, NXDOMAIN, NOTIMP, REFUSED and other numeric codes.
Large TXT/NULL	Examines answer records of types 16 and 10; flags a string representation longer than 120 characters.

Returned evidence

The result includes total_dns_queries, unique_domains, response_codes, nxdomain_count, record_types and the total suspicious-name count. It returns up to 30 flagged domains sorted by subdomain entropy, 15 top names by frequency and 10 large TXT/NULL records. Flagged records include the name, selected subdomain, both entropy values, length and explanatory reasons.

Question counting applies whenever a DNS packet contains a question, including responses that echo it. total_dns_queries therefore counts observed question sections, not exclusively client requests. NXDOMAIN is reported as a count; no failure-ratio threshold is implemented.

The large-record field length_bytes is calculated from len(str(rdata)), so it is the length of a rendered value rather than the original wire payload. High entropy and long labels are investigation signals; legitimate encoded identifiers can exhibit the same properties.

7 Threat heuristics

scan_suspicious_indicators evaluates raw payloads and TCP metadata. Findings include a category, severity, protocol, source, destination, packet_index and detail. Packet-specific indices are one-based; aggregate SYN-scan findings use zero.

Signal	Rule and assigned severity
HTTP Basic authentication	Case-insensitive Authorization: Basic pattern; Base64-decodes the value. HIGH.
HTTP POST secrets	POST-like payload containing password, passwd, pass, pwd, api_key, apikey, secret or token assignments. MEDIUM.
FTP USER / PASS	Line-based USER and PASS patterns in raw text. USER is HIGH; PASS is CRITICAL.
Telnet interaction	login: or password: text with TCP source or destination port 23. MEDIUM insecure-protocol observation.
TCP SYN scanning	Exact SYN-only flags; at least 15 distinct destination ports from one source across the capture. HIGH.

Periodic traffic analysis

TCP timestamps are grouped by source IP, destination IP and destination port. Source port is not part of this key. Every TCP packet contributes, including packets without application payload. A group needs at least eight observations before timing is assessed.

Successive capture-order differences form the interval series. The analyzer computes the mean interval, population variance and standard deviation. It reports a potential beacon when the mean exceeds 0.5 seconds and the ratio of standard deviation to mean is below 0.2.

Beacon records contain source, destination and port, connection_count, interval_mean_seconds and jitter_ratio. connection_count is the number of recorded TCP packets in the group, not a count of established sessions. Periodic benign activity can satisfy the same timing rule.

Evidence interpretation

total_findings and the CRITICAL, HIGH and MEDIUM counters cover the findings list. potential_c2_beaconing is a separate list and is not included in those totals. The SYN rule aggregates the whole capture rather than applying a sliding time window or confirming handshake failure.

Credential inspection decodes each raw packet independently as UTF-8, ignoring invalid bytes. It does not reconstruct split application messages first. FTP patterns are not constrained to port 21, and the Telnet rule detects prompt interaction rather than recovering a complete login exchange. Extracted secrets remain visible in returned detail strings.

8 TLS fingerprints, filters and streams

TLS ClientHello analysis

The TLS parser examines raw TCP payloads beginning with a handshake record byte (0x16) and ClientHello type (1). It reads the ClientHello version, cipher suites, extension types, supported groups, point formats and SNI. Parsing operates per packet without TCP reassembly or TLS decryption.

The parser removes values present in its GREASE_VALUES set and computes an MD5 digest over a constructed string. The code joins all five fingerprint groups using hyphens, and its GREASE set contains 0xBAAB rather than 0xBABA. Consequently the returned ja3_hash is an implementation-specific fingerprint and should not be assumed compatible with standard JA3 intelligence feeds. No JA4 calculation is implemented.

Results include handshake count, distinct fingerprints and SNIs, top-ten frequency lists, and up to 50 handshake records with timestamp, source, destination, SNI, hash and ClientHello version. The version describes the parsed ClientHello field, not a verified negotiated TLS version. No malware attribution lookup is performed.

Native display filters

TShark discovery checks TSHARK_PATH, then PATH, then common Windows, Linux and macOS locations. apply_display_filter invokes TShark with -r, -n, optional -Y and -T fields. It returns packet number, time, source, destination, protocol, length and info, with limit/offset metadata. Pagination slices the collected output after execution; it does not limit TShark's scan.

Native filtering uses a 45-second subprocess timeout. Other wrapper commands default to 30 seconds. Arguments are passed as a list without a shell. Overview optionally adds the io,phs hierarchy and suppresses failures of that supplementary step. Native conversation and stream methods exist in the wrapper but are not used by their corresponding MCP tools.

Stream transcripts

follow_stream groups payload-bearing TCP or UDP packets by sorted endpoint pairs. Stream indices follow the first occurrence of each pair, so they are not guaranteed to match Wireshark stream numbers. The transcript retains capture order and labels direction and packet number. Although TCP sequence numbers are recorded internally, they do not drive ordering, deduplication or gap recovery.

Payloads are decoded as UTF-8 with replacement characters. The 65,536-byte default threshold is checked after appending a complete chunk, so returned content can exceed it. total_stream_bytes counts processed chunks; total_packets_in_stream counts all payload-bearing chunks in the selected pair. Results include endpoints, protocol, truncated and stream_content.

9 Installation and operation

The package requires Python 3.10 or later. Runtime dependencies are mcp $\geq$ 1.0.0, scapy $\geq$ 2.5.0 and pydantic $\geq$ 2.0.0. requirements.txt also includes pytest $\geq$ 8.0.0. TShark is optional for the Scapy-based tools.

Clone the repository, create a virtual environment and install the package from its root directory. The commands below use a POSIX shell; on Windows, activate .venv\Scripts\Activate.ps1 instead.

```
git clone https://github.com/ritvikindupuri/Wireshark_MCP.git
cd Wireshark_MCP
python -m venv .venv
source .venv/bin/activate
python -m pip install -e .
python -m wireshark_mcp.server
```

Installing the package also exposes the wireshark-mcp command. The process communicates over stdio; it is not a browser dashboard or HTTP service. Configure the MCP client to launch the virtual environment's Python interpreter.

```
{
  "mcpServers": {
    "wireshark": {
      "command": "/absolute/path/.venv/bin/python",
      "args": ["-m", "wireshark_mcp.server"]
    }
  }
}
```

The example assumes the package was installed into that environment. For a source-only launch, set PYTHONPATH to the repository root through the client environment. Set TSHARK_PATH to the executable when discovery through PATH or standard locations is unsuitable.

Typical investigation

Begin with get_engine_status, then pcap_overview and list_conversations for the capture. Use DNS, TLS and threat tools to examine specific evidence. Request a stream transcript or native display filter when the initial results identify a relevant flow. For Base64 data, analyze_uploaded_pcap combines the main triage steps into one call.

Resource and data handling

Scapy analyzers load captures with rdpcap and reread the file for separate analyses. Only uploads have a 50 MiB limit; direct file inputs have no equivalent cap. Local paths are checked for file existence, not restricted to an allowlisted directory. Process filesystem permissions define access.

Returned capture contents can contain credentials and attacker-controlled text. There is no application-level redaction, prompt-injection filter, authentication layer or retention scheduler. The MCP host controls how tool evidence reaches a model. Running the process with access only to intended captures keeps that boundary explicit.

10 Source reference and next steps

This document describes the executable package and its supporting configuration at the revision below. The code review covers tool registration, analyzer logic, engine routing and output contracts. It does not claim that the application or its tests were executed.

Repository: https://github.com/ritvikindupuri/Wireshark_MCP

Revision: e8803288d888f7306cd3a8f68f2f1fb1a013e2c7

Source	Documented behavior
wireshark_mcp/server.py	Ten MCP tools, upload policy, server instructions and sequential triage.
engines/detector.py	TShark executable discovery.
engines/scapy_engine.py	Overview fields and bidirectional conversation aggregation.
engines/tshark_engine.py	Subprocess arguments, timeouts, filtering and native statistics helpers.
analyzers/dns.py	Entropy, question/response counts and TXT/NULL inspection.
analyzers/heuristics.py	Credential patterns, SYN scanning and timing calculations.
analyzers/tls.py	ClientHello parsing, SNI and fingerprint construction.
analyzers/streams.py	Payload grouping, transcript order and truncation.
pyproject.toml / requirements.txt	Package metadata, executable entry point and dependencies.
tests/	Synthetic capture generators and analyzer/server assertions.

Paths beneath engines and analyzers are relative to wireshark_mcp. The test suite contains 15 test functions across test_server.py and test_analyzers.py. They exercise sample-capture outputs, including upload and combined triage, but do not establish production coverage or standard JA3 compatibility.

Next steps

Priorities for deeper forensic interoperability are standards-compatible fingerprint serialization, a corrected GREASE set and fixture comparisons against trusted TLS implementations.

Sequence-aware TCP reassembly would improve segmented credential inspection and stream reconstruction.

For larger captures, incremental reading and bounded result sizes would reduce repeated memory allocation. For shared operation, add explicit capture-directory restrictions, unique upload names, retention controls and optional secret redaction. These are proposed extensions, not components shown as implemented in the diagrams.

Diagram conventions

The architecture uses the official Wireshark icon for TShark and the supplied agent icon for the external AI assistant. Scapy processing and forensic analyzers are ordinary software components. Model providers, cloud services and container infrastructure are not implied by the diagram.